BugIt
by Taskivator

الدليل الكامل خطوة بخطوة

BugIt

وكيل ضمان الجودة لإنشاء تذاكر الخلل وإرسالها إلى نظام التتبع عبر VS Code

الإعداد لأول مرة والاستخدام اليومي: دليل تفصيلي يشرح كل نقرة، ومكتوب لمن لم يسبق لهم القيام بأي شيء مماثل. لا تحتاج إلى خلفية تقنية. اتّبعه بالترتيب، وستتمكن اليوم من إنشاء تذاكر خلل واضحة ومنظّمة وإرسالها إلى نظام التتبع.

يعمل في: VS Code على Windows (الأساسي)، وmacOS، وLinux

تنويه بشأن الترجمة.

تُرجم هذا المستند آليًا ولم يخضع لمراجعة متحدثين أصليين باللغة. النسخة الإنجليزية هي المرجع المعتمد: وعند وجود أي اختلاف يُعتد بالنص الإنجليزي. وللاطلاع على أدق صياغة وأحدثها، يُرجى الرجوع إلى المستند الإنجليزي.

محتويات الدليل

نفّذ **الجزء 1** مرة واحدة وبالترتيب. بعد ذلك، يمكنك الرجوع إلى أي جزء كلما احتجت إليه. يجيب قسم **استكشاف الأخطاء وإصلاحها** و**الأسئلة الشائعة** قرب نهاية الدليل عن أكثر الأسئلة شيوعًا؛ يُرجى الاطلاع عليه قبل مراسلة الدعم عبر البريد الإلكتروني.

الجزء 1 · الإعداد (نفّذ هذا مرة واحدة)

الخطوة 0: اختر مساعد الذكاء الاصطناعي الخاص بك

الخطوة 1: ثبّت VS Code

الخطوة 2: فك ضغط ملف BugIt الذي نزّلته

الخطوة 3: افتح BugIt في VS Code

الخطوة 4: اختر وكيل BugIt في الدردشة

الخطوة 5: فعّل ترخيصك

الخطوة 6: اقبل الشروط (مرة واحدة)

الخطوة 7: شغّل `Begin setup`

الخطوة 8: صِل نظام التتبّع الخاص بك

الخطوة 9: احفظ بيانات تسجيل الدخول

الخطوة 10: تأكّد من جاهزيتك

الجزء 2 · الاستخدام اليومي

الأوامر في لمحة

الجزء 3 · خصّصه ليناسبك

الجزء 4 · التحديثات والنسخ الاحتياطية والسلامة

الجزء 5 · استكشاف الأخطاء وإصلاحها والأسئلة الشائعة

الأسئلة الشائعة

تراخيص `Team` : أسئلة

الجزء 6 · الترخيص والدعم

القاعدة الوحيدة التي تحافظ على سلامتك

لا ينشئ BugIt **أبدًا** أي تذكرة ولا يرسلها من دون أن يعرض عليك معاينة ويحصل أولًا على تأكيدك المكتوب `FILE IT` كرسالة كاملة. ولا تكفي كلمة «yes» وحدها. أنت المتحكّم دائمًا. هل تريد التدرّب بلا أي مخاطرة؟ اكتب `dry run`، وسيكتب التقرير كاملًا **من دون** إنشاء تذكرة أو إرسالها.

الإعداد لأول مرة مقابل الاستخدام اليومي

لن تنفّذ **الجزء 1** كاملًا إلا مرة واحدة. بعد ذلك، يصبح فتح BugIt سريعًا، ولا تتطلب التحديثات سوى أمر واحد (`Update`). راجع الجزء 4.

يستغرق نحو 15 دقيقة. نفّذ الخطوات بالترتيب؛ فكل خطوة تعتمد على سابقتها. لا تتجاوز أي خطوة.

0 اختر مساعد الذكاء الاصطناعي الخاص بك

يتبع هذا الدليل مسار GitHub Copilot

BugIt هو «السائق»، والذكاء الاصطناعي هو «المحرّك» الذي يكتب التقارير فعليًا. تهيّئك هذه الخطوات لاستخدام GitHub Copilot داخل VS Code، وهو أسرع مسار إذا كنت تبدأ من الصفر. يعمل BugIt أيضًا مع إضافة Claude، ومع أي مساعد آخر يستطيع تنفيذ أمر، وبمفرده باستخدام مفتاح الذكاء الاصطناعي الخاص بك. إذا كنت تستخدم أحدها بالفعل، فانتقل إلى الخطوة 2 وافتح مجلد BugIt في تلك الأداة بدلًا من VS Code.

قبل أن تبدأ: تحتاج إلى أحدث إصدار من VS Code وإلى مساعد يستطيع تنفيذ أمر، مثل GitHub Copilot Chat أو إضافة Claude.

ما هو؟ GitHub Copilot مساعد يعمل بالذكاء الاصطناعي من GitHub، المملوكة لشركة Microsoft. يوفّر مستوى مجانيًا للاستخدام الخفيف ومستوى مدفوعًا للاستخدام المكثف. إذا اخترته، فستثبّته في الخطوة 1 وتسجّل الدخول.

1. ستحتاج إلى **حساب GitHub**. إذا لم يكن لديك حساب، فأنشئ حسابًا مجانيًا على `github.com/signup` .
2. يكفي المستوى المجاني من Copilot لتجربة BugIt. إذا كنت تسجّل بلاغات أخطاء طوال اليوم، فإن خطة `Copilot Pro` المدفوعة تستحق تكلفتها؛ ويمكنك الترقية في أي وقت على `github.com/features/copilot` .
3. ستفعّل Copilot فعليًا داخل VS Code في الخطوة 1؛ ولا تحتاج إلى فعل أي شيء آخر هنا الآن.

1 ثبّت VS Code

VS Code تطبيق مجاني من Microsoft. وهو «النافذة» التي يعمل BugIt داخلها.

1. افتح متصفح الويب وانتقل إلى `code.visualstudio.com` .
2. انقر زر `Download` الأزرق الكبير؛ فهو يكتشف نظامك تلقائيًا.
3. افتح الملف الذي نزّلته وثبّته؛ **اقبل جميع الخيارات الافتراضية**، وواصل النقر على `Next/Install` .
4. افتح VS Code بعد تثبيته.

فعّل GitHub Copilot داخل VS Code

1. في أقصى يسار VS Code، انقر أيقونة `Extensions` ، التي تبدو كأربعة مربعات صغيرة.
2. في مربع البحث، اكتب `GitHub Copilot` .
3. انقر `Install` على `GitHub Copilot` ، وكذلك على `GitHub Copilot Chat` إذا ظهر.
4. ستظهر مطالبة تطلب منك اختيار `Sign in to GitHub` ؛ انقر هذا الخيار وسجّل الدخول إلى حسابك على GitHub في نافذة المتصفح التي تُفتح.

ستعرف أنه نجح عندما: تظهر أيقونة Copilot صغيرة أسفل VS Code، وتفتح لوحة `Chat` ؛ وهي أيقونة فقاعة الكلام على اليسار، أو يمكنك الضغط على `Ctrl+Alt+I` .

2 فك ضغط ملف BugIt الذي نزّلته

تضمّن شراؤك ملفًا بامتداد `.zip` ، مثل `bugit-qa-agent.zip` . عليك فك ضغطه؛ فلن يعمل إذا شغّلته وهو مضغوط.

1. ابحث عن ملف `.zip` ، وعادةً ما يكون في مجلد `Downloads` .
2. **انقر عليه بزر الفأرة الأيمن** ← اختر `Extract All…` في Windows، أو انقر عليه نقرًا مزدوجًا في Mac.
3. اختر مكانًا بسيطًا مثل مجلد `Documents` ، ثم انقر `Extract` .
4. سيصبح لديك الآن مجلد **BugIt** يحتوي على ملفات كثيرة. تذكّر مكانه.

لا تضعه في OneDrive أو في مجلد متزامن

قد تسبب مجلدات المزامنة السحابية تعارضات في الملفات. مجلد `Documents` أو `Desktop` مناسب تمامًا. كذلك لا تغيّر أسماء الملفات داخله؛ اترك المجلد كما هو.

3 افتح BugIt في VS Code

1. في VS Code، انقر `File → Open Folder` من القائمة العلوية اليسرى.
2. انتقل إلى المكان الذي فككت فيه ضغط BugIt، مثل `Documents` .
3. انقر مجلد **BugIt** مرة واحدة لتحديده، ثم انقر `Select Folder` .
4. إذا سألك VS Code: `Do you trust the authors of the files in this folder?` ، فانقر `Yes, I trust the authors` .

ستعرف أنه نجح عندما: ترى ملفات BugIt مدرجة على الجانب الأيسر من VS Code، ومنها مجلدات مثل `tools` و `.github` وملفات مثل `README.md` .

4 اختر وكيل BugIt في الدردشة

1. افتح لوحة Copilot Chat؛ انقر أيقونة فقاعة الكلام على اليسار، أو اضغط `Ctrl+Alt+I` في Windows أو `Cmd+Alt+I` في Mac.
2. توجد أعلى مربع الدردشة قائمة منسدلة صغيرة للأوضاع، وقد تعرض `Ask` أو `Agent` .
3. انقر عليها واختر `BugIt QA Agent` من القائمة.

ألا ترى `BugIt QA Agent` في القائمة؟

تأكّد من أنك فتحت مجلد BugIt في الخطوة 3، لا مجلدًا مختلفًا؛ فالوكيل يُحمّل من داخله. أغلق المجلد وأعد فتحه إذا لزم الأمر.

5 فعّل ترخيصك

كل ما تحتاج إليه هو حساب BugIt؛ فلا يوجد مفتاح ترخيص لتنسخه أو تلصقه. توافق على هذا الجهاز في متصفحك، ثم يُكمل الأمر `Activate` الباقي.

1. في مربع الدردشة، اكتب `hi` واضغط `Enter` .
2. اكتب `Activate` . يفتح BugIt موقع **BugIt Portal** في متصفحك. سجّل الدخول إلى حسابك الشخصي، واختر استحقاق `Solo` أو `Team` ، ووافق على هذا الجهاز. ثم عُد إلى VS Code؛ يكمل BugIt التفويض تلقائيًا. تبقى كلمة مرورك في المتصفح ولا تُدخل أبدًا في VS Code.

> **ستعرف أنه نجح عندما:** يؤكّد BugIt التفعيل ويذكر الحساب الذي سجّلت الدخول به.

> **حافظ على خصوصية حسابك**
>
> حسابك والاستحقاق المرتبط به خاصان بك. لا تشاركهما أو تنشرهما أو تعيد بيعهما؛ فقد يُلغى الوصول المشترك. يسجّل كل عضو في `Team` الدخول بحسابه الخاص؛ فلا يوجد مفتاح مشترك ولا بيانات دخول مشتركة. ولا بأس بالانتقال إلى كمبيوتر جديد لاحقًا؛ راجع الأسئلة الشائعة.

6 اقبل الشروط (مرة واحدة)

في المرة الأولى، يعرض BugIt ملخصًا قصيرًا ويطلب منك القبول قبل أن يفعل أي شيء آخر.

1. اقرأ الملخص القصير الذي يعرضه: تراجع كل تذكرة قبل إنشائها وإرسالها؛ وتقع مسؤولية سلامة مشروعك عليك؛ وتُعد إجراءات الحماية وسائل مساعدة وليست ضمانات. ولأن نموذج الذكاء الاصطناعي الذي يجيبك في Copilot هو الذي يتبع هذه الإجراءات، فقد يتجاوز نموذج أخف أو مجاني خطوة أحيانًا، أو يحتاج إلى تكرار أمر، في حين لا يحتاج نموذج أقوى إلى ذلك.
2. اختر **I agree** من الخيارين اللذين يعرضهما BugIt.

> **ستعرف أنه نجح عندما:** يؤكد BugIt ذلك وينتقل إلى الإعداد. لن تُسأل عن هذا إلا مرة واحدة؛ فهو يتذكر.

7 شغّل `Begin setup`

يُهيّئ BugIt نفسه الآن من خلال طرح أسئلة عليك بلغة واضحة. لن تحتاج أبدًا إلى تعديل ملف إعدادات يدويًا.

أنت تكتب

```
Begin setup
```

سيطرح أسئلة قصيرة ولن يفعّل إلا ما تختاره. وأهم سؤال يتعلق بنظام تتبّع العيوب الذي تستخدمه.

8 صِل نظام التتبّع الخاص بك

سجّل الإعداد أي نظام تتبّع تستخدمه. وهذه الخطوة هي ما يتيح لـBugIt أن ينشئ التذاكر فيه فعليًا: تنشئ رمز API في حسابك الخاص، ثم تحفظه على هذا الجهاز بأمر واحد. ينشئ BugIt التذاكر مباشرةً عبر واجهة REST API الخاصة بنظام التتبّع باستخدام هذا الرمز، فلا شيء يحتاج إلى البقاء قيد التشغيل، وإنشاء التذاكر لا يستخدم خادم MCP إطلاقًا.

8a · أنشئ رمزًا في حساب نظام التتبّع الخاص بك

نظام التتبع لديك	أين تنشئها
Jira Cloud	`id.atlassian.com/manage-profile/security/api-tokens`
Azure DevOps	`dev.azure.com/YOUR-ORG/_usersSettings/tokens`
GitHub Issues	`github.com/settings/personal-access-tokens/new`
GitLab Issues	`gitlab.com/-/user_settings/personal_access_tokens`
Bugzilla	في Bugzilla الخاصة بك: API Keys → Preferences
YouTrack	في YouTrack الخاص بك: الصورة الرمزية → Profile → Account Security → Authentication
Linear	`linear.app/settings/account/security`
Shortcut	`app.shortcut.com/settings/account/api-tokens`
ClickUp	`app.clickup.com/settings/apps`
Asana	`app.asana.com/0/my-apps`
Trello	`trello.com/apps/admin` (مفتاح API ورمز تصرّح به بواسطته)

> **انسخ الرمز فور ظهوره**
>
> لا تعرض معظم الخدمات الرمز الجديد إلا مرة واحدة. انسخه فورًا واحتفظ به في مكان آمن إلى أن يُحفظ في المخزن الآمن على جهازك (الخطوة 9). اختر أطول مدة صلاحية يتيحها نظام التتبّع حتى لا تضطر إلى تكرار ذلك قريبًا.

8b · احفظه على هذا الجهاز

اطلب من مساعدك توصيل نظام التتبّع لديك فيقوم بذلك نيابةً عنك. ولتفعل ذلك بنفسك، افتح طرفية داخل VS Code وشغّل الأمر الخاص بنظام التتبّع لديك، مستبدلًا `jira` بـ `ado` أو `github` أو `gitlab` وما إلى ذلك.

أنت تشغّل (في الطرفية)

```
python tools/connect.py jira
```

8c · الصق الرمز في الطلب المُخفّى

يطلب الأمر الرمز عبر طلب محلي مُخفّى: فلا يظهر على الشاشة ولا يصل أبدًا إلى الدردشة أو إلى تذكرة أو إلى `config.json` . الصقه هناك ولا شيء غير ذلك. بعدها يسجّل BugIt الدخول، ويسرد المشاريع التي يراها الرمز فعلًا، ويقرأ أنواع القضايا المسموح لك بإنشائها، و**يرفض حفظ بيانات اعتماد لا تستطيع إنشاء التذاكر**. لذلك فإن اكتمال الإعداد يعني أن الإنشاء يعمل.

8d · تحقق من اتصاله

للتأكد في أي وقت، شغّل الفحص في الطرفية، أو اسأل في الدردشة ببساطة:

أنت تشغّل (في الطرفية)

```
python tools/connect.py jira --check
```

أنت تكتب

```
Check status
```

ستعرف أن العملية نجحت عندما: يعرض BugIt نظام تتبّع العيوب بالحالة `connected / healthy` .

لا شيء يحتاج إلى التشغيل بعد إغلاق VS Code

يوجد رمز API الخاص بك في مخزن بيانات الاعتماد في نظام التشغيل، لا داخل VS Code. لذلك لا يوجد شيء تشغّله عند إعادة فتح VS Code: يستطيع BugIt إنشاء التذاكر فورًا. وللتأكد في أي وقت، شغّل `python tools/connect.py jira --check` .

9 احفظ بيانات تسجيل الدخول

بعد حفظ الرمز (الخطوة 8c)، يحميه جهازك بنفسه، ولذلك لن تحتاج إلى البحث عن الرمز مرة أخرى: على Windows يكون سجلًا مشفّرًا لا يفتحه إلا حساب Windows الخاص بك على ذلك الجهاز، وعلى macOS وLinux يوجد في سلسلة المفاتيح أو في خدمة الأسرار. ولا يحتوي `config.json` إلا على المؤسسات وعناوين URL، ولا تصل قيم بيانات الاعتماد أبدًا إلى سجل الدردشة أو إلى ملف نصي عادي.

لمعرفة أنظمة التتبّع التي لديها بيانات اعتماد محفوظة، اكتب:

أنت تكتب

```
Check saved credentials
```

سيعرض أنظمة التتبّع التي لديها بيانات اعتماد محفوظة، ولن **يعرض أبدًا قيم بيانات الاعتماد**. وإذا أُزيلت بيانات اعتماد أو انتهت صلاحيتها، فشغّل الأمر نفسه مرة أخرى (الخطوة 8b) ليحل الرمز الجديد محل القديم.

---

10 تأكّد من جاهزيتك

اطلب من BugIt التحقق من كل شيء مرة أخرى قبل إنشاء أول تذكرة حقيقية:

أنت تكتب

```
Check readiness
```

سيعرض أي شيء لا يزال ناقصًا (وعادةً لا يتجاوز ربط نظام التتبع أو التفعيل). وعندما تصبح جميع المؤشرات خضراء، سترى ✅ `"Setup complete."`

وبذلك يكتمل الإعداد، نهائيًا

لن تحتاج إلى تكرار الجزء 1. ومن الآن فصاعدًا، كل ما يلزم لفتح BugIt هو: افتح المجلد ← اكتب `hi` . ولتغيير أي اختيار لاحقًا، ما عليك سوى كتابة `Begin setup` وذكر ما تريد تغييره.

ألا يتوفر GitHub Copilot؟ يوجد معالج في الطرفية

إذا لم تتمكن من استخدام Copilot، فافتح الطرفية المدمجة (`Terminal → New Terminal`) وشغّل `python tools/setup_wizard.py`. سيطرح بضعة أسئلة ويكتب إعداداتك من دون الذكاء الاصطناعي. ويمكنك أيضًا تشغيل BugIt بصورة مستقلة باستخدام مفتاح ذكاء اصطناعي خاص بك؛ راجع الأسئلة الشائعة.

الجزء 2 · الاستخدام اليومي

بعد إتمام الإعداد، ستنجز عملك بالكامل من خلال الدردشة. تحدّث بأسلوب طبيعي، أو استخدم الأمثلة المحددة أدناه. اكتب `help` في أي وقت لعرض القائمة الكاملة.

كتابة تقرير خلل

هذه هي المهمة الرئيسية لـBugIt. صِف الخلل بجملة بسيطة؛ فيكتب التذكرة كاملة، ويملأ الإصدار والفئة تلقائيًا، ويتحقق من وجود تذاكر مكررة، ويعرض عليك معاينة، ثم ينشئها ويرسلها إلى نظام التتبع فقط بعد أن ترد بـ `FILE IT` كرسالة كاملة. أما «نعم» وحدها فتُرفَض.

أنت تكتب

```
Write a bug report: the app crashes every time I tap Save on iPhone
```

يُعدّ مسودة تتضمن عنوانًا، وخطوات إعادة إنتاج الخلل، والنتيجة المتوقعة مقابل النتيجة الفعلية، ودرجة الخطورة، والمنصة. ويسجل درجة الخطورة التي تؤكدها في نص التذكرة. وإذا كان نظام التتبع والمشروع يوفران حقلًا أصليًا للأولوية أو درجة الخطورة، فإنه يضبط ذلك الحقل أيضًا. ولا يتوفر مثل هذا الحقل في GitHub أو GitLab أو Trello. هل تريد إجراء تغييرات قبل إنشاء التذكرة وإرسالها؟ ما عليك سوى قول ذلك، مثل: «اجعل درجة الخطورة مرتفعةً» أو «أضف أنه بدأ في أحدث إصدار».

البلاغات السريعة

بالنسبة إلى الأنماط الشائعة (`crash` و `error` و `missing` و `layout` و `slow` و `stuck` و `data` و `visual` و `audio` و `blocker`)، يتجاوز النموذج السريع بعض الأسئلة ويملأ الفئة والأولوية نيابةً عنك، ويتخطى البحث عن التذاكر المكررة لتوفير الرحلة إلى نظام التتبع. استخدمه عندما تعرف مسبقًا أن المشكلة جديدة؛ فالمعاينة تذكر ذلك، ولا يزال التقرير العادي يجري التحقق الكامل من التكرار.

أنت تكتب (أمثلة)

```
Quick bug: crash on startup after update
Quick bug: layout button overlaps text on the settings screen
Quick bug: blocker can't continue past the payment step
Quick bug: slow the dashboard takes 20 seconds to load
```

تقارير التعطل

إذا كنت قد ربطت أداةً لتتبّع الأعطال، مثل Sentry، فسينفّذ BugIt كل ما سبق، بالإضافة إلى ربط تقريرك بحادثة التعطل وجلب تتبع المكدس نيابةً عنك.

أنت تكتب

```
Write a crash bug report: game crashes when opening the map
```

تعليقات التحقق والإغلاق

بعد اختبار إصلاح، يكتب BugIt تعليقًا منظمًا لنشره على التذكرة. وهو يكتب التعليق، وينشره اختياريًا، لكنه لا يغيّر حالة التذكرة. يظل عليك إغلاق التذكرة أو حلّها أو إعادة فتحها بنفسك في نظام التتبع.

ما تكتبه	ما تحصل عليه
`Close #123`	يسألك عن السيناريو المقصود، مثل: تأكيد الإصلاح، أو الإغلاق بناءً على طلب، أو الإغلاق لأن السلوك مطابق للتصميم، أو إعادة الفتح؛ ثم يكتب التعليق المناسب.
`Write a verification comment for the build you tested on Windows`	تعليق «تم تأكيد الإصلاح» مع ملء تفاصيل الإصدار.
`Write a reopen comment for the build you tested on Windows`	تعليق «ما زالت المشكلة تحدث»، ثم تعيد فتح التذكرة بنفسك.

الترجمة

ترجم التقارير أو المصطلحات بين اللغات التي أعددتها، باستخدام الصياغات الدقيقة لفريقك من المسرد؛ راجع الجزء 3.

أنت تكتب (أمثلة)

```
Translate to ja: the save button does nothing on the profile screen
Translate this bug report to English: [paste text]
```

البحث عن المعلومات: المواصفات والمسرد

إذا كنت قد ربطت المواصفات أو ملأت المسرد، فاطرح على BugIt أسئلةً عن مشروعك.

أنت تكتب (أمثلة)

```
What is [your term]?
Walk me through the checkout flow
What's new in specs?
```

اكتشاف التذاكر المكررة

يحدث هذا تلقائيًا في التقرير العادي، قبل كتابة أي شيء. يبحث BugIt في نظام التتبع عن التذاكر المطابقة، حتى إن كانت مصاغة بطريقة مختلفة قليلًا، ويحذّرك حتى لا تقدّم الخلل نفسه مرتين. أما النموذج السريع فيتخطى البحث عمدًا لتوفير الرحلة إلى نظام التتبع، ويذكر ذلك في المعاينة، فتعرف دائمًا أيًّا منهما حصلت عليه.

الميزات المتقدمة، إذا فعّلتها

اكتب هذا	ما يفعله
`Triage digest`	ملخص فوري للاجتماع اليومي: الأخطاء المفتوحة حسب المجال/الخطورة · مع الإشارة إلى الأقدم.
`Export bug to file`	يحفظ التقرير بصيغة Markdown/CSV/JSON بدلًا من إرساله إلى نظام التتبع، وهو خيار ممتاز قبل إعداد نظام التتبع.
`Undo last filing`	يعرض آخر عملية كتابة في سجل التدقيق. لا يستطيع BugIt حذف تذكرة أو إغلاقها؛ وبعد معاينة جديدة وكتابة FILE IT يمكنه إزالة تعليق أو مرفق أنشأه هو فقط.
(تلقائيًا)	علامات SLA على الأخطاء العاجلة · حقول خاصة بكل فئة.

قائمة الأوامر الكاملة

أنت تكتب

```
help
```

...يعرض كل أمر داخل الوكيل، في أي وقت.

> **تدرّب بأمان في أي وقت**
> اكتب `dry run` وسيكتب BugIt التقرير كاملًا **من دون** إنشاء أي تذكرة أو إرسالها، وهو مثالي للتعلّم أو الاختبار.

الأوامر في لمحة

يمكنك دائمًا أن تصف ما تريده بكلماتك أنت. وهذه هي الصيغ التي يتعرّف عليها BugIt إن كنت تفضّلها.

تكتب هذا	ماذا يحدث
إنشاء تذكرة	
`Quick bug: [type] [subject]`	إنشاء سريع. يتخطى البحث عن التكرارات عمدًا، ويذكر ذلك في المعاينة.
`dry run`	يكتب التقرير كاملًا ولا يُنشئ شيئًا. الطريقة الآمنة للتدرّب.
`FILE IT`	الشيء الوحيد الذي يُنشئ التذكرة. يُكتب رسالةً كاملة، مرة واحدة، للمسودة التي قرأتها للتو.
`Export bug to file`	يحفظ التقرير ملفًا بدل إرساله إلى نظام التتبع.
`Undo last filing`	يعرض آخر عملية كتابة، ويمكنه إزالة تعليق أو مرفق أضافه BugIt. ولا يستطيع إغلاق تذكرة. يتطلب تفعيل سجل التدقيق.
`Triage digest`	ملخّص لما أنشأته مؤخرًا.
معرفة مشروعك	
`What is [term]?`	يبحث عن مصطلح في مسردك.
`What's new in specs?`	التغييرات الأخيرة في مجلد المواصفات الذي دللت BugIt عليه.
`Look up crash [ID]`	يبحث في أداة الأعطال لديك ويعيد الروابط فقط. ولا يُكتب أي تقرير.
`Look up #[ticket]`	يبحث في نظام التتبع لديك ويعيد الروابط فقط. ولا يُكتب أي تقرير.
`Seed glossary: [text]`	يقترح مصطلحات للمسرد من نص تعطيه إياه، وتوافق أنت على كل مصطلح.
`Learn from my recent tickets`	يتعلّم أسلوبك من التذاكر الموجودة أصلًا في نظام التتبع لديك.
الإعداد والفحص	
`Begin setup`	إعداد أول كامل، أو تغيير خيار واحد لاحقًا.
`Check status`	هل أدواتي متصلة؟
`Check readiness`	ما الذي ينقص قبل أن أتمكن من الإنشاء؟
`Add a custom MCP server`	يوصّل أداة غير مضمّنة أصلًا.
`Fix servers`	يشخّص اتصالات MCP ويصلحها.
`Check saved credentials`	الأسماء فقط. لا يعرض BugIt أبدًا قيمة بيانات اعتماد محفوظة.
`Talk to me in [language]`	يغيّر اللغة التي يحادثك بها BugIt على هذا الجهاز. وهي منفصلة عن لغة كتابة تقاريرك.
`help`	النسخة المختصرة من هذا الجدول، داخل المحادثة.

تكتب هذا	ماذا يحدث
الترخيص والتحديثات والنسخ الاحتياطية	
`Activate`	يفوّض هذا الجهاز عبر المتصفح. ولا يُلصق أي مفتاح أو كلمة مرور في المحرر إطلاقًا.
`License status`	خطتك واستحقاقك وحالتك دون اتصال.
`What version am I on?`	إصدارك الحالي، وهل هناك تحديث في الانتظار.
`Update`	يثبّت أحدث إصدار موقّع في مكانه. تبقى إعداداتك كما هي، ويُنسخ لها احتياطيًا أولًا.
`Back up my settings`	يحفظ محليًا الإعدادات والمسرد وأسلوب الكتابة وإعداد MCP. ولا تُدرج بيانات الاعتماد.
`List backups`	يعرض النسخ الاحتياطية التي يمكنك الرجوع إليها.
`Restore my settings`	يرجع إلى إحداها.
`Create support bundle`	ملف تشخيص منقّى لإرفاقه برسالة إلى الدعم. بلا بيانات اعتماد ولا تذاكر ولا مسارات.
Team	
`Authorize this Team device`	يعتمد هذا الجهاز على حسابك أنت في BugIt، عبر المتصفح.
`Manage my devices`	يفتح BugIt Portal، حيث يُسجَّل خروج الجهاز ويتحرر مقعده.
`Manage my Team`	يفتح BugIt Portal، حيث تُدار الأعضاء والأدوار والمقاعد والتراخيص.

الجزء 3 · اجعله خاصًا بك: أضف معرفة مشروعك

يأتي BugIt في الأصل كوكيل عام لضمان الجودة من دون معرفة مسبقة بمشروعك. إليك كيفية تعليمه تفاصيل مشروعك بعد الشراء. لا تحتاج إلى البرمجة؛ فمعظم العمل يتم ببساطة عبر التحدث إليه في الدردشة. كل ما تضيفه يبقى على جهازك.

1 · كلمات فريقك: المسرد

يجعل ذلك الترجمات واستنتاج الفئات أدقّ وأكثر ملاءمةً لمشروعك.

1. افتح الملف `.github/glossary/terms.template.md` من قائمة الملفات.
2. املأ الجدول بكلمات فريقك: أسماء الميزات، وأسماء الشاشات، وأسماء العناصر، والاختصارات، وترجماتها إذا كنت تستخدم لغتين.
3. احفظ الملف باستخدام `Ctrl+S` . هذا كل شيء؛ سيستخدم BugIt هذه المصطلحات الدقيقة من الآن فصاعدًا.

اختصار

فعّل «البذر التلقائي للمسرد» أثناء `Begin setup` ، وسيقترح BugIt مصطلحات مأخوذة من تذاكرك الحالية؛ وكل ما عليك فعله هو الموافقة على كل مصطلح.

2 · أسلوب كتابة تقارير الخلل المعتمد لدى فريقك

هل تريد أن تتبع التذاكر تنسيق فريقك بدقة، بما في ذلك أسلوب العناوين، وتسميات الخطورة، والحقول المطلوبة؟ لا **تصفه** له، بل **اعرضه** عليه:

1. قل في الدردشة: «طابق أسلوب فريقي».
2. الصق **لقطة شاشة واحدة** لتذكرة موجودة في نظام التتبع.
3. يقرأ الصورةَ نموذجُ الذكاء الاصطناعي الموجود أصلًا في محادثتك، وفق شروط مزوّده، ثم يحجب BugIt البيانات الشخصية الشائعة في النص المستخرج منها بأفضل جهد ممكن، ويحفظ تنسيقك، ثم يتبعه في كل تذكرة مستقبلية.

3 · فئاتك ومنصاتك وحقولك

ما عليك سوى إخبار BugIt بلغة طبيعية، وسيحدّث إعداداتك نيابةً عنك:

أنت تكتب (أمثلة)

```
My categories are Gameplay, UI, Audio, and Networking
We test on Windows and Xbox
```

4 · أدواتك الخاصة: الاتصالات المخصصة

هل تستخدم أداةً غير موجودة في القائمة المدمجة؟ اربطها بالطريقة نفسها التي تربط بها أداةً مدمجة:

أنت تكتب

```
Add a custom MCP server
```

امنحها اسمًا قصيرًا وعنوانها، الذي يجب أن يبدأ بـ `https://` ، وسيتصل بها BugIt ويتحقق من سلامتها نيابةً عنك.

5 · صحّح له ببساطة أثناء العمل

هذه أسهل طريقة على الإطلاق: عندما يُعدّ BugIt مسودة تذكرة، صحّح أي شيء لا يعجبك، سواء كان تسميةً أو صياغةً أو مستوى خطورة. عند تفعيل `learn from your edits` ، وهو الخيار الموصى به، يتذكر تفضيلاتك ويطبّقها في المرة التالية. تتراكم معرفة مشروعك بصورة طبيعية على جهازك، ولا يُرسَل أيٌّ منها إلى Taskivator أبدًا.

كل ما تضيفه يظل ملكك

يُحفظ مسردك وأسلوب الكتابة المعتمد لدى فريقك وفئاتك والتفضيلات التي تعلّمها BugIt على جهازك، وتُنشأ نسخة احتياطية منها قبل كل تحديث. وعند تنفيذ BugIt لطلبك، قد يُرسَل السياق ذو الصلة إلى نموذج الذكاء الاصطناعي والأدوات التي تربطها أنت. ولا يُرسَل أي من ذلك إلى Taskivator، ولا تغادر حزمة المشاركة جهازك إلا عندما تختار إرسالها.

6 · مشاركة إعدادك مع فريقك: ترخيص Team

بعد إعداد المسرد، وأسلوب الفريق، وخيارات نظام التتبع بالطريقة التي تريدها، شارك هذا الإعداد مع بقية فريقك بدلًا من تكرار الخطوات 1 - 3 على كل جهاز:

ما تشغّله مرةً واحدة، بعد أن يصبح إعدادك كما تريده

```
python tools/share.py export
```

ينشئ ذلك الملف `config.share.zip` ، الذي يجمع خيارات نظام التتبع وأداة الأعطال والاتصالات، والمسرد، وأسلوب الفريق، من دون أي كلمات مرور أو رموز وصول داخله. أرسله إلى فريقك.

ما يشغّله كل زميل أولًا، للمراجعة

```
python tools/share.py import config.share.zip
```

يعرض هذا ما سيتغيّر ولا يكتب شيئًا.

بعد المراجعة، لتطبيق الإعدادات

```
python tools/share.py import config.share.zip --apply
```

يطبّق هذا إعدادات نظام التتبع وأداة الأعطال والاتصالات، من دون إعادة كتابة الفئات. ولتفعيل المسرد وأسلوب الفريق أيضًا، عليهم مراجعة تلك التعليمات ثم اختيار `python tools/share.py import config.share.zip --apply --apply-instructions` بدلًا من ذلك. تبقى تراخيصهم وإعدادات أجهزتهم الخاصة كما هي، ولا تُنقل أي بيانات اعتماد ضمن الحزمة.

إذا غيّر الاستيراد وجهة إرسال البيانات

إذا كان الإعداد المشترك سيضيف اتصالًا مخصصًا جديدًا أو عنوانًا للإشعارات، تعرض المعاينة بدقة ما سيتغيّر قبل تطبيق أي شيء. كما ينشئ التطبيق أولًا لقطةً لإعداد الزميل الحالي، بحيث يمكن استخدام `Restore my settings` للتراجع إذا لم تكن النتيجة كما توقع.

الجزء 4 · التحديثات والنسخ الاحتياطية والسلامة

الحصول على التحديثات (بأمر واحد)

عندما يتوفر إصدار أحدث، يذكره BugIt مرة واحدة عندما تقول `hi` . تحديثات البرنامج مشمولة ما دام ترخيص BugIt الخاص بك ساريًا. لتثبيته:

أنت تكتب

```
Update
```

1. ينشئ BugIt أولًا نسخة احتياطية جديدة من ملفاتك.

2. ينزّل الإصدار الجديد ويتحقق من أنه أصلي ولم يُعبث به (فهو موقّع تشفيريًا، ويُرفض أي تحديث مزيف أو معدّل).
3. يثبّته من دون المساس بإعداداتك، أو مسرد مصطلحاتك، أو أسلوب فريقك، أو ترخيصك، أو رموز الوصول الخاصة بك.
4. يطلب منك إعادة تحميل VS Code (`Ctrl+Shift+P` ← اكتب `Reload Window` ← `Enter`). ابدأ محادثة جديدة وستكون على الإصدار الجديد.

> **لا حذف للمجلدات، إطلاقًا**
>
> على خلاف التثبيت الأول، تتم التحديثات في مكانها. لا تحذف أي شيء أو تعيد تنزيله مطلقًا، وتظل تهيئتك محفوظة دائمًا وتُنشأ لها نسخة احتياطية.

ما الذي يمكن تعديله يدويًا بأمان، وما الذي لا يمكن؟

آمن، ويُحتفظ به خلال كل تحديث: `config.json` ، و `.github/instructions/house-style.instructions.md` (راجع الجزء 3، فهذه هي الطريقة المدعومة لتخصيص السلوك)، و `.github/glossary/terms.template.md` ، و `.github/instructions/learned.instructions.md`، و `.vscode/mcp.json` .

> **غير آمن: لا تعدّل هذه الملفات يدويًا**
>
> ملف الوكيل (`.github/agents/bugit-qa-agent.agent.md`)، وأي ملف آخر ضمن `.github/instructions/` ، وأي شيء في `tools/` ، كلها أجزاء من المنتج نفسه وليست إعداداتك. **يستبدلها التحديث بصمت**، وعلى خلاف الملفات أعلاه، **لا توجد نسخة احتياطية يمكنك استعادتها منها**. يتحقق BugIt من ذلك عند بدء التشغيل ومرة أخرى قبل تثبيت أي تحديث مباشرة، ويحذّرك إذا وجد تغييرًا، لكن القاعدة الأكثر أمانًا هي ببساطة ألا تعدّلها.

النسخ الاحتياطية والاستعادة

ما تكتبه	ما يفعله
`Back up my settings`	يحفظ لقطة محلية من إعداداتك، ومسرد مصطلحاتك، وأسلوب فريقك، وإعدادات نقاط نهاية MCP، وما تعلّمه النظام. يُستبعد استحقاق جهازك الموقّع وقيم بيانات الاعتماد.
`List backups`	يعرض كل لقطة محفوظة مع تاريخها.
`Restore my settings`	يعيد الحالة إلى لقطة محفوظة، وينشئ أولًا لقطة لحالتك الحالية، بحيث يمكن التراجع عن عملية الاستعادة نفسها.

> **تلقائيًا قبل كل تحديث**
>
> ينشئ BugIt دائمًا نسخة احتياطية قبل تثبيت إصدار جديد، ولذلك لن يؤدي أي تحديث إلى فقدان إعداداتك. إذا بدت إعداداتك غير صحيحة بعد تحديث، يعيدها `Restore my settings` إلى وضعها السليم.

السلامة: ما الذي تحميه

لا شيء من دون موافقتك

تُعرض معاينة لكل تذكرة وتعليق؛ وأي إجراء لا يمكن التراجع عنه يتطلب أن يكون ردك هو `FILE IT` بالضبط، بوصفه رسالتك كاملة، ومرتبطًا بالمسودة التي قرأتها للتو، وقابلًا للاستخدام مرة واحدة.

`dry run` = وضع المحاكاة

قل `dry run`، تُمنَع كل تذكرة أو تعليق أو تعديل أو مرفق أو إزالة أو إشعار خارجي (أما الأوامر المحلية التي تشغّلها بنفسك، مثل النسخ الاحتياطي أو التصدير، فقد تنشئ ملفات محلية)، ولا يتم الاتصال بنظام التتبع إطلاقًا، بما في ذلك القراءة: لا بحث، ولا تحقق من التكرار، ولا فحص للاتصال، لأن كلًا منها يصل إلى لوحتك ويفتح الرمز المحفوظ لديك. والمتغير `QA_AGENT_DRY_RUN=1` في الطرفية هو المفتاح الذي تفرضه الأوامر المرفقة في الشيفرة؛ أما قول « `dry run` » في المحادثة فهو طلب من المساعد بالتوقف، وهو مفيد لكنه ليس الضمانة نفسها.

لا أسرار في الملفات

تظل رموز الوصول في خزنة نظام التشغيل لديك، ولا توضع أبدًا في ملف نصي عادي، ولا تُرسل إلينا أبدًا.

يعمل على جهازك

لا يصل عملك إلا إلى الأدوات التي تربطها به ونموذج الذكاء الاصطناعي الخاص بك.

بياناتك وخصوصيتك

الشيء الوحيد الذي يرسله BugIt إلى Taskivator هو بيانات الترخيص والتحديث: تسجيل الدخول إلى حساب BugIt الخاص بك (في المتصفح، على Portal)، ومعرّف جهاز مجزّأ أحادي الاتجاه (رمز مموّه لا يكشف هويتك)، ومعرّف تثبيت عشوائي، واسم جهازك (اسم المضيف) واسم نظام التشغيل لديك، وإصدار BugIt ومرشّح الخطة الذي تختاره، ومادة تحقق تشفيرية قصيرة العمر، ورمز موافقة لمرة واحدة مأخوذ من رابط Portal الذي فتحه متصفّحك، وسر إقرار لكل تفعيل (تجزئته، والقيمة نفسها فقط عند سحب وصول هذا الجهاز)، واستحقاق `Solo` أو `Team` الذي توافق على استخدامه لهذا الجهاز. لا يوجد مفتاح ترخيص، وتظل كلمة مرورك في المتصفح؛ فلا تُدخل أبدًا في VS Code. لا تُرسل تقارير الخلل، أو المواصفات، أو مسرد المصطلحات، أو الإعدادات، أو رموز الوصول إلى Taskivator مطلقًا. أما ما ترفعه أنت فيذهب إلى حيث أرسلته: إلى نظام التتبع الخاص بك، في حسابك أنت، برمز الوصول الخاص بك. وإذا استخدمت نموذج ذكاء اصطناعي مستضافًا، فإن مسودتك تصل إلى مزوّده أيضًا. لا قياس عن بُعد ولا تحليلات، أبدًا. ترد التفاصيل الكاملة في ملف `PRIVACY.md` داخل مجلد BugIt.

المخاطر والتنبيهات المهمة: يرجى قراءتها

قد يخطئ الذكاء الاصطناعي

يصوغ BugIt التقارير باستخدام نموذج ذكاء اصطناعي قد يسيء فهم التفاصيل أو يختلق شيئًا. اقرأ المعاينة دائمًا قبل أن ترد `FILE IT`. فأنت توافق على ما سيُنشأ ويُرسل.

ينشئ التذاكر ويرسلها إلى نظام التتبع الحقيقي لديك

بمجرد موافقتك، تُنشأ تذكرة حقيقية. هل تتدرّب؟ استخدم `dry run` لكتابة التقرير من دون إرساله.

سلامة مشروعك مسؤوليتك

تقع مسؤولية كيفية استخدامك BugIt وسلامة مشاريعك وبياناتك وأنظمة التتبع لديك عليك. تقلّل وسائل الحماية المخاطر، لكنها لا تحل محل مراجعتك.

ما لا يفعله BugIt

يسجّل BugIt في أنظمة التتبع الأحد عشر التي يذكرها جميعًا، كلٌّ منها ببيانات اعتماد تنشئها في حسابك الخاص، ويتحقق BugIt منها مقابل الوجهة التي تختارها قبل حفظها. وما يصل إلى حقل أولوية قابل للفرز يختلف من نظام إلى آخر، ولا يوجد حقل من هذا النوع في GitHub وGitLab وTrello، فتُكتب الخطورة هناك داخل نص التذكرة. وConfluence وSentry وNotion للقراءة فقط؛ ولا يُحتاج خادم MCP متوافق تُعدّه أنت إلا لها، أما إشعارات Slack وTeams وDiscord والبريد فمدمجة ولا تحتاج إلى خادم، بمساعدة BugIt. ويستخدم ذكاءك الاصطناعي (Copilot أو مفتاح Anthropic/OpenAI الخاص بك) لا نموذجًا مرفقًا؛ وإخفاء البيانات يجري ببذل أفضل جهد؛ ويعمل في VS Code وفي امتداد Claude وفي طرفية عادية. ترخيص واحد = جهاز واحد (Solo) أو 5 مقاعد أعضاء بجهاز واحد لكل عضو (Team)؛ ويمكن للفريق الواحد أن يحمل أكثر من ترخيص Team، فتتجمع مقاعدها. وتعتمد النتائج واتساقها أيضًا على نموذج الذكاء الاصطناعي المُجيب: فالنموذج الأقوى يتبع خطوات الأمان بموثوقية أعلى من نموذج خفيف جدًا أو مجاني.

الجزء 5 · استكشاف الأخطاء وإصلاحها والأسئلة الشائعة

عند حدوث أي خلل، صِفه أولًا للمساعد بلغة بسيطة. يمكن تشخيص معظم مشكلات الإعداد والاستخدام اليومي داخل المحرر مباشرةً.

جرّب هذا أولًا: اطلب من الذكاء الاصطناعي إصلاح المشكلة

كلما حدث خطأ، سواء أثناء الإعداد أو الاستخدام اليومي، يكون أسرع حل في الغالب هو **سؤال المساعد مباشرة في المحادثة**. صف ما حدث بكلمات بسيطة، مثل «توقف الإعداد في منتصفه» أو «لا يتصل نظام التتبّع الخاص بي»، واطلب منه إصلاحه. يستطيع الذكاء الاصطناعي تشخيص معظم المشكلات وإصلاحها في الحال. يرجى تجربة ذلك قبل مراسلتنا؛ فهو يحل الغالبية العظمى من المشكلات خلال ثوانٍ.

تطبيق إصلاح نفّذه الذكاء الاصطناعي: زر `Keep`

إذا أصلح المساعد شيئًا بتغيير ملف، يعرض VS Code شريطًا صغيرًا يحتوي على `Keep` / `Undo`. إذا كنت قد طلبت الإصلاح ورضيت عنه، فانقر على `Keep`؛ وسيُطبَّق التغيير على نسختك الخاصة فقط، على جهازك. انقر على `Undo` لتجاهله. لا تتم مشاركة أي شيء تختار له `Keep` مع أي شخص آخر.

ما تراه	ما ينبغي فعله
`"Activate to start"`	اكتب `Activate`. يفتح BugIt موقع BugIt Portal في متصفحك؛ سجّل الدخول، واختر استحقاق `Solo` أو `Team`، ووافق على هذا الجهاز. ألا تملك حسابًا بعد؟ احصل على حساب من المتجر.

ما تراه	ما ينبغي فعله
لم يُفتح المتصفح، أو لم يكتمل التفعيل	اكتب `Activate` مرة أخرى لإعادة فتح BugIt Portal، ثم أكمل تسجيل الدخول والموافقة على هذا الجهاز. اترك الأمر يعمل حتى يخبرك بأن التفويض حُفظ على هذا الحاسوب. ولا يوجد مفتاح للصقه.
`"No tracker enabled"`	اكتب `Begin setup` واختر نظام التتبع لديك.
يُرفض إنشاء التذكرة	شغّل `python tools/connect.py jira --check` . سيخبرك بمن تكون لدى نظام التتبع، وبما يحق لـBugIt إنشاؤه، ويعرض ضمن `fixes` العلاج المحدد.
يواصل مطالبتك بالتفعيل	اكتب `Activate` وأكمل تسجيل الدخول في المتصفح. لا يعرض BugIt أبدًا قيم بيانات الاعتماد المحفوظة. هل هذه هي المرة الأولى؟ راجع الخطوة 8c.
لا ينشئ تذكرة الخلل ولا يرسلها	اكتب `Check readiness` ؛ وسيعرض بالتحديد ما ينقصك، وغالبًا ما يكون السبب أن رمز نظام التتبّع لم يُحفظ بعد.
يبدو شيء ما معطلًا بعد تحديث	إذا تغيّرت إعداداتك، فاكتب `Restore my settings` . وإذا كانت ملفات BugIt نفسها تالفة، فاكتب `Repair my BugIt install` وراجع المعاينة أولًا. وإذا كانت الملفات سليمة وظل الإصدار الجديد يعمل بشكل غير صحيح، فأرسل بريدًا إلكترونيًا إلى support@bugit.dev.
يبدو أن الوكيل «لا يلتزم بالتعليمات»	اكتب `Read and follow all your given instructions` ، أو ابدأ محادثة جديدة. قد يحدث هذا بوتيرة أكبر مع نموذج ذكاء اصطناعي خفيف أو مجاني؛ ويكون النموذج الأقوى في منتقي النماذج في Copilot أكثر اتساقًا.
تبدو الإجابات عامة أو غير مخصصة للمشروع	زوّده بمزيد من السياق، أو اعرض عليه تذكرة حالية أثناء `Begin setup` ليتعلّم أسلوبك. أعد تشغيل `Begin setup` لتحسينه.
تريد تغيير أحد خيارات الإعداد	اكتب `Begin setup` واذكر ما تريد تغييره، مثل `"add a tracker"` أو `"start over"` ؛ وسيعيد فتح ذلك الجزء فقط. إذا كان الإعداد قد انقطع في منتصفه فحسب، فإن كتابة `Begin setup` وحدها تستأنف من موضع التوقف بدلًا من البدء من جديد.

> **فحوص السلامة المضمّنة**
>
> `Check status` (هل أدواتي متصلة؟) · `Check readiness` (ما المتبقي قبل أن أستطيع إنشاء تذكرة وإرسالها؟). لإجراء فحص أعمق، افتح `Terminal → New Terminal` وشغّل `python tools/selftest.py` .

الأسئلة الشائعة

هل أحتاج إلى معرفة البرمجة؟

لا. إذا كنت تستطيع تثبيت تطبيق وكتابة جملة، فيمكنك استخدام BugIt. فهو يتولى الأجزاء التقنية نيابة عنك.

هل سينشئ تذاكر خلل ويرسلها من دون علمي؟

أبدًا. تُعرض معاينة لكل تذكرة وتعليق أولًا، ولا يحدث أي إجراء لا يمكن التراجع عنه إلا عندما يكون ردك هو `FILE IT` بالضبط. تتم مطابقته بوصفه رسالتك كاملة، ولذلك لا يترتب أي إجراء على جملة تَرِد فيها العبارة عرضًا. وهو مرتبط بالمسودة نفسها

التي قرأتها، ولذلك يطلب BugIt موافقتك من جديد إذا حدث تغيير لاحق. كما يُستخدم مرة واحدة، فلا يمكن لأمر مكرر أن ينشئ التذكرة ويرسلها مرتين. لا يستطيع BugIt أبدًا حذف تذكرة، أو تغيير حالتها، أو تغيير المكلّف بها. للتدرّب، قل `dry run`؛ تُمنَع كل تذكرة أو تعليق أو تعديل أو مرفق أو إزالة أو إشعار خارجي (أما الأوامر المحلية التي تشغّلها بنفسك، مثل النسخ الاحتياطي أو التصدير، فقد تنشئ ملفات محلية)، وسيرفض الوكيل عمليات الكتابة إلى نظام التتبع، مع أن هذا الرفض يعتمد على تعليمات BugIt لا على قفل تفرضه المنصة أو نظام التشغيل، لذلك استخدم بيانات اعتماد للقراءة فقط عند التقييم.

هل يجب أن أشغّل خادمًا في كل مرة؟

لا. يستخدم إنشاء التذاكر رمز API الذي حفظته بالأمر `python tools/connect.py`، فلا يوجد شيء تشغّله. ولا يلزم خادم MCP إلا للتكاملات الاختيارية للقراءة فقط مثل Confluence وSentry وNotion.

هل يمكنني استخدامه على حاسوبين؟

يمكن استخدام جهاز واحد في كل مرة مع `Solo`. أما في `Team`، فلكل عضو حسابه الخاص وجهازه الخاص، ويأتي عدد المقاعد من تراخيص `Team` التي يملكها الفريق: كل ترخيص `Team` يضيف 5 مقاعد. لا بأس بالانتقال إلى جهاز جديد: نشّط الجهاز الجديد، وسيتوقف الجهاز القديم فورًا عن القدرة على تجديد تفويضه. يتحرر مقعده بمجرد أن يعيد الجهاز القديم الاتصال، مؤكدًا أنه تخلّى عن الوصول، أو عند انتهاء فترة التفويض الحالية للعمل دون اتصال البالغة 72 ساعة؛ ولذلك لا توجد فترة انتظار ثابتة. اكتب `License status` للتحقق.

ماذا يحدث إذا تعطل خادم الترخيص؟

يمكنك مواصلة العمل. بعد تفعيل ناجح عبر الإنترنت، يعمل BugIt بموجب تفويض مخزّن محليًا لمدة تصل إلى 72 ساعة من دون اتصال، ولذلك لن تقاطعك أعطال الخادم، أو حتى عطلة نهاية أسبوع بلا إنترنت، خلال تلك المدة. بعد ذلك يعيد التحقق عبر الإنترنت.

هل أحتاج إلى GitHub Copilot؟

لا. يعمل BugIt في أي مساعد يستطيع تنفيذ أمر: Copilot Chat أو إضافة Claude في VS Code أو طرفية عادية. Copilot هو أسهل طريقة. إذا لم يكن متوفرًا لديك، شغّل `python tools/setup_wizard.py` في `Terminal` لإجراء الإعداد، ويمكن لـBugIt العمل بصورة مستقلة باستخدام مفتاح Anthropic/OpenAI الخاص بك عبر `python standalone/run.py`.

هل بياناتي خاصة؟

نعم. لا يرسل BugIt إلى Taskivator سوى بيانات الترخيص والتحديث: تسجيل الدخول إلى حساب BugIt (في المتصفح، على Portal)، ومعرّف جهاز مجزّأ أحادي الاتجاه، ومعرّف تثبيت عشوائي، واسم جهازك (اسم المضيف) واسم نظام التشغيل، وإصدار BugIt ومرشّح الخطة الذي تختاره، ومادة تحقق تشفيرية قصيرة العمر، ورمز موافقة لمرة واحدة مأخوذ من رابط Portal الذي فتحه متصفّحك، وسر إقرار لكل تفعيل (تجزئته، والقيمة نفسها فقط عند سحب وصول هذا الجهاز)، واستحقاق `Solo` أو `Team` الذي توافق عليه لهذا الجهاز. لا يستخدم برنامج BugIt قياس Google Ads ولا يرسل أي بيانات تتبّع للمنتج. لا تذهب تذاكرك نفسها إلا إلى نموذج الذكاء الاصطناعي ونظام التتبع اللذين تربطهما، ولا تصل إلينا أبدًا.

هل يمكنني تعديل ملفات BugIt؟

يمكنك تخصيص مسرد المصطلحات وأسلوب الكتابة المعتمد لدى فريقك والإعدادات الواردة في الجزء 3. لكن لا تعطّل نظام الترخيص أو التفعيل. إذا واجهت خللًا حقيقيًا في الأداة نفسها، فراسل الدعم حتى يمكن إصلاحه للجميع في التحديث التالي.

مع أي أنظمة تتبع للعلل يعمل؟

أحد عشر، ويسجّل BugIt فيها جميعًا: Jira Cloud وAzure DevOps وGitHub Issues وGitLab Issues وBugzilla وYouTrack وLinear وShortcut وClickUp وAsana وTrello. ولكلٍّ منها إعداد موجَّه، وكلٌّ منها يكتب ببيانات اعتماد

تنشئها في حسابك الخاص، ويتحقق BugIt منها مقابل وجهتك قبل حفظها. وما يختلف هو الإضافات التي تصل: لا يرفع BugIt الملفات إلى GitHub أو GitLab أو Bugzilla ويخبرك بذلك قبل المحاولة (فـGitHub لا يوفر رفعًا برمز وصول إطلاقًا، بينما يوفره GitLab وBugzilla ولم يعتمده BugIt)، ويقبل ClickUp الملف ولا يستطيع إزالته، ولا يوجد حقل أولوية في GitHub وGitLab وTrello، فتُكتب خطورتك هناك داخل نص التذكرة. اختر نظامك أثناء الإعداد وبدّله في أي وقت بالأمر `Begin .setup`

هل سيكتشف تقارير الخلل المكررة قبل أن أرسلها؟

نعم، في التقرير العادي. قبل كتابة أي شيء، يبحث في نظام التتبع لديك عن تقارير مشابهة، حتى إن كانت صياغتها مختلفة قليلًا، ويعرض لك النتائج المطابقة كي تتجنب تسجيل الخطأ نفسه مرتين. ويظل قرار المتابعة لك. أما النموذج السريع فيتخطى ذلك البحث عمدًا لتوفير الرحلة إلى نظام التتبع، وتذكر المعاينة ذلك.

هل يمكنه كتابة التذاكر بأكثر من لغة؟

نعم. أضف لغة ثانية أثناء الإعداد، وسيُكتب كل تقرير باللغتين في كتلتين منفصلتين، مع استخدام مسرد مصطلحاتك كي تظل مصطلحات المنتج دقيقة؛ فهو لا يرتجل ترجمة أبدًا.

هل يمكنني إرفاق لقطة شاشة؟

اطلب من BugIt إرفاقها. سينشئ مجلد أدلة ويفتحه ويخبرك بمساره الدقيق؛ ضع ملفاتك فيه ثم أخبره بأنك انتهيت. لا يمكن لـ BugIt الوصول إلى ملف مُلصق في المحادثة، ولذلك يعرض عليك المجلد دائمًا. يفحص BugIt الملفات النصية بحثًا عن بيانات اعتماد ويتوقف بدل إرفاقها، ويعرض عليك معاينة قبل الإرسال. ولا يستطيع النظر داخل الصور، لذا تحقق من لقطة الشاشة بنفسك قبل الموافقة عليها. ويجري الإرفاق بعد موافقتك.

هل يخفي كلمات المرور ورموز الوصول في تقاريري؟

عند تشغيل حجب البيانات الحساسة، يحجب بأفضل جهد ممكن عناوين البريد الإلكتروني ورموز الوصول وعناوين IP في كل مسودة قبل إرسالها. وترى دائمًا النص الكامل في المعاينة أولًا.

أنشأت تذكرة وأرسلتها عن طريق الخطأ؛ هل يمكنني التراجع عنها؟

تمنع المعاينة وكتابة FILE IT معظم الأخطاء قبل كتابة أي شيء. إذا كنت قد فعّلت سجل التدقيق، فاكتب `Undo last filing`: فهو يعرض ما كتبه BugIt ويمكنه سحب تعليق أو مرفق أضافه BugIt نفسه. أما إغلاق التذكرة أو حذفها فهو دائمًا إجراؤك أنت في نظام التتبع لديك، لأن أيًا من الأنظمة لا يتيح لـBugIt طريقة لفعل ذلك.

هل يمكنه مساعدتي في التحقق من تقرير خلل أو إغلاقه؟

نعم. اطلب تعليق تحقق، وسيصوغ تعليقًا واضحًا يبيّن نجاح التحقق أو فشله، بلغاتك، على التذكرة الحالية؛ وتوافق عليه قبل نشره.

هل يمكنني استخدامه لأكثر من مشروع؟

يُعدّ كل مجلد BugIt للعمل مع نظام تتبّع واحد لمشروع واحد. لمشروع آخر، إما أن تعيد تشغيل `Begin setup` لتوجيهه إلى مكان جديد، أو تحتفظ بنسخة منفصلة لكل مشروع.

كيف أحصل على التحديثات؟

عند صدور إصدار جديد، يذكره BugIt عند بدء المحادثة. اكتب `update` وسيثبّته في مكانه؛ مع الاحتفاظ بمسرد مصطلحاتك، وأسلوب فريقك، وإعداداتك، وإنشاء نسخة احتياطية منها أولًا.

ماذا يحدث عند انتهاء ترخيصي؟

عند انتهاء مدتك، جدّد من حسابك؛ وسيطبّق جهازك الاستحقاق المجدّد في المرة التالية التي يتحقق فيها عبر الإنترنت. فترة التفويض للعمل دون اتصال البالغة 72 ساعة والمذكورة أعلاه منفصلة؛ فهي لا تغطي إلا انقطاعًا مؤقتًا لخادم الترخيص، ولا تغطي انتهاء مدة الترخيص. اكتب `License status` في أي وقت لمعرفة وضعك.

إذا جدّدت ترخيصي أو اشتريت ترخيصًا آخر، فهل تتراكم الأيام؟

الأيام لا تتراكم أبدًا. يحل التجديد **محل** استحقاقك الحالي، وتبدأ مدته من جديد من يوم سريانه، ولا تُرحّل الأيام غير المستخدمة. لذلك جدّد قرب نهاية مدتك الحالية، لا مبكرًا. **أما المقاعد فمختلفة.** في `Team` ، يضيف الترخيص الثاني مقاعده الخمسة إلى المقاعد الأولى، ولكل ترخيص مدته الخاصة: أي إن ترخيصين يمنحانك 10 مقاعد. راجع أسئلة `Team` أدناه.

تراخيص `Team` : أسئلة

كل ما في هذه الصفحة يجري في **بوابة BugIt** داخل متصفحك، في صفحة `Team` ، لا في المحرر. وإذا اشتريت ترخيص `Solo` فيمكنك تخطي هذه الصفحة.

ماذا أحصل عليه مع ترخيص `Team` ؟

خمسة مقاعد لمدة سنة، ترخيص لمدة سنة بلا اشتراك ولا يتجدد تلقائيًا. يسجّل كل عضو في الفريق الدخول بحسابه الخاص في BugIt وينشّط BugIt على جهاز خاص به: لا يوجد مفتاح مشترك ولا تسجيل دخول مشترك. أما الأعضاء والدعوات والأدوار والمقاعد فتُدار كلها في صفحة `Team` في البوابة.

كيف أضيف شخصًا؟

افتح صفحة `Team` وأرسل دعوة إلى عنوان بريده الإلكتروني؛ ويمكنك لصق عدة عناوين دفعة واحدة. لا تصلح كل دعوة إلا للعنوان الذي أُرسلت إليه، فلا ينفع رابط مُعاد توجيهه شخصًا آخر. والدعوة المعلّقة تحجز مقعدًا حتى تُقبل أو تُلغى، ولهذا قد يمتلئ الفريق قبل انضمام الجميع.

من يستطيع أن يفعل ماذا؟

المالك يستطيع كل شيء، بما في ذلك الفوترة وتسليم الفريق إلى شخص آخر. و**المشرف** يستطيع دعوة الأعضاء وإزالتهم وتغيير الإعدادات المشتركة للفريق. و**العضو** يستخدم BugIt ويمكنه المغادرة متى شاء. ولا تُنزع الملكية، بل تُنقل.

نحتاج أكثر من خمسة أشخاص. هل يمكنني شراء ترخيص `Team` آخر؟

نعم. اشترِ ترخيص `Team` ثانيًا وأضفه إلى الفريق نفسه من صفحة `Team` . **وتتجمع المقاعد:** ترخيصان يمنحانك 10 مقاعد، وثلاثة تمنحك 15. ولا يوجد حد أقصى ثابت، لأن كل مقعد يأتي من ترخيص تملكه أنت. وتسري الإضافة فورًا.

إذا أضفت ترخيص `Team` ثانيًا، فمتى تنتهي صلاحية فريقي؟

يحتفظ كل ترخيص بتاريخ انتهائه الخاص، ويدفع مقاعده الخمسة حتى ذلك التاريخ. ويظل فريقك يعمل ما دام **أي** من تراخيصه ساريًا، فيكون تاريخ انتهاء الفريق هو الأبعد من التاريخين. وعند انتهاء الترخيص الأسبق يفقد الفريق المقاعد الخمسة التي كان يدفعها ذلك الترخيص، لكن الفريق نفسه لا يُغلق. ولا يمدّد الترخيص الثاني الأول: فإذا بقي على أحدهما ثلاثة أشهر وأضفت ترخيصًا جديدًا لسنة كاملة، صار لديك 10 مقاعد لثلاثة أشهر ثم 5 مقاعد للأشهر التسعة التالية. جدّد الترخيص الأسبق للإبقاء على المقاعد العشرة كلها. وتعرض صفحة `Team` كل ترخيص بتاريخ انتهائه الخاص، فترى دائمًا أيها ينتهي أولًا.

ماذا يحدث إذا صار عدد الأشخاص أكبر من عدد المقاعد؟

نراسل الفريق قبل انتهاء أي ترخيص بثلاثين يومًا وبسبعة أيام وبيوم واحد، فلا يكون الأمر مفاجئًا أبدًا. وإذا انتهى ترخيص فعلًا والفريق يشغل مقاعد أكثر مما يدفع، تتخلى الدعوات المعلّقة عن مقاعدها أولًا، ثم يفقد الوصول من انضموا حديثًا. ولا يُزال المالك أبدًا، فيبقى دائمًا من يستطيع التجديد. ولا يُحذف شيء مما سُجّل؛ إنما يتوقف الوصول إلى BugIt فحسب.

ماذا لو غادر الشخص الذي يملك فريقنا؟

يستطيع المالك ترشيح **خلَف** في صفحة `Team` . فإذا تعذّر الوصول إلى المالك بعد ذلك، طلب الخلَف تولّي الفريق؛ ويُبلَّغ جميع أعضاء الفريق، ويسري التغيير بعد 7 أيام ما لم يوقفه المالك قبل ذلك. أما المالك المتاح فيستطيع نقل الفريق في أي وقت، ويُطلب منه أولًا التأكيد بالمصادقة الثنائية.

هل يمكن للفريق كله أن يعمل بالطريقة نفسها؟

نعم. اضبط BugIt كما تريد، ثم نفّذ `python tools/share.py export` ، وينفّذ كل زميل `import` على الملف الذي ترسله إليه (الجزء 3، القسم 6). كما تسجّل صفحة `Team` نظام التتبع والمشروع والمسؤول والوسوم التي اتفق عليها فريقك، ليطّلع عليها الجميع في مكان واحد. **ولا تضع هناك أبدًا رمزًا مميزًا أو كلمة مرور:** فكل من في الفريق يستطيع قراءة هذه الإعدادات. أما بيانات الاعتماد الحقيقية فتُحفظ دائمًا لكل شخص على حدة بأمر `python tools/connect.py` على جهازه هو.

غادر أحدهم الفريق. ماذا يحدث لمقعده؟

أزِله من صفحة `Team` : يُسجَّل خروج جهازه ويتحرر المقعد فورًا لدعوتك التالية. ويمكن للعضو أيضًا أن يغادر بنفسه. وفي الحالتين يبقى حسابه في BugIt ملكًا له، وينتهي وصوله إلى فريقك فقط.

إلى أين تصل الإيصالات وإشعارات التجديد؟

إلى عنوان حساب المالك بشكل افتراضي. ويستطيع المالك في صفحة `Team` إضافة اسم شركة وبريد فوترة منفصل، فتذهب إشعارات ذلك الفريق وإيصالاته إليه. ولا يغيّر ذلك من يملك الفريق.

الجزء 6 · الترخيص والدعم

شروط الترخيص (ملخص بلغة مبسطة)

ترد الشروط الكاملة والملزمة في ملف `LICENSE` داخل مجلد BugIt، وتلك الوثيقة هي الحاكمة. باختصار:

الموضوع	بلغة مبسطة
مرخّص، وليس مبيعًا	تشتري ترخيصًا لاستخدام BugIt، ولا تشتري ملكيته. تحتفظ Taskivator بجميع الحقوق.
مقاعدك	`Solo` = جهاز واحد في كل مرة. و `Team` = 5 مقاعد أعضاء لكل ترخيص `Team` ، بجهاز واحد لكل عضو، ويمكن لأكثر من ترخيص `Team` أن يموّل الفريق نفسه. حسابك واستحقاقك خاصان بك؛ فلا تشاركهما أو تبيعهما من جديد أو تنشرهما.
إلغاء الصلاحية	يمكن إلغاء الوصول إذا تمت مشاركته، أو رُدَّ ثمن الترخيص، أو أُلغيت الدفعة باعتراض مصرفي، أو أُسيء استخدامه.

الموضوع	بلغة مبسطة
خصّصه، ولكن…	عدّل إعداداتك، ومسرد مصطلحاتك، وقوالبك بحرية، لكن لا تعطّل نظام الترخيص أو التفعيل ولا تتحايل عليهما.
مسؤوليتك	تقع مسؤولية كيفية استخدامك BugIt وسلامة مشروعك عليك. تراجع كل تقرير وتوافق عليه قبل إنشاء التذكرة وإرسالها. وسائل الحماية أدوات مساعدة وليست ضمانات.
«كما هو»، والمسؤولية	يُقدَّم كما هو، من دون ضمان. وفي حدود ما يسمح به القانون، تقتصر المسؤولية على المبلغ الذي دفعته، مع استبعاد الأضرار غير المباشرة أو التبعية.
المدة والقانون	ترخيص لمدة سنة واحدة تبدأ في يوم شرائه؛ وهو بلا اشتراك ومن دون تجديد تلقائي، وتحكمه قوانين اليابان. تُعالج عمليات استرداد الأموال من خلال المتجر الذي اشتريت منه.
الأيام لا تتراكم، والمقاعد تتراكم	يحل التجديد محل استحقاقك الحالي؛ وتبدأ مدته من جديد، ولا تُرحّل الأيام غير المستخدمة، لذلك جدّد قرب نهاية مدتك. وفي `Team` ، يضيف كل ترخيص إضافي مقاعده طوال مدته الخاصة؛ ويظل الفريق يعمل حتى انتهاء آخر ترخيص ساري لديه. عند انتهاء المدة، جدّد من حسابك، وسيُعاد تفويض جهازك عند اتصاله التالي بالإنترنت للتحقق.

وثيقتان في مجلدك

`LICENSE` (الشروط الكاملة) و `PRIVACY.md` (بيان الخصوصية). بتنشيط BugIt واستخدامه، فإنك تقبل `LICENSE` .

الحصول على المساعدة

يرجى مراجعة قسمي استكشاف الأخطاء وإصلاحها والأسئلة الشائعة أعلاه أولًا؛ فهما يغطيان كل شيء تقريبًا. ثم اتبع هذا الترتيب:

1 · اطلب من الذكاء الاصطناعي إصلاح المشكلة

أفضل خطوة أولى: صف المشكلة في المحادثة واطلب من المساعد إصلاحها. إذا غيّر ملفًا وبدا التغيير صحيحًا، فانقر على `Keep` لتطبيقه على حاسوبك فقط.

2 · شغّل فحصًا للحالة

`Check status` · `Check readiness` · أو `python tools/selftest.py` في `Terminal` .

3 · الاستعادة

يعيد `Restore my settings` إعداداتك ومسرد مصطلحاتك وأسلوب فريقك، لكنه لا يلغي أي تحديث. لملفات BugIt التالفة: `Repair my BugIt install` .

4 · تواصل مع شخص

فقط إذا لم يساعدك الدليل والذكاء الاصطناعي: زُر bugit.dev أو أرسل بريدًا إلكترونيًا إلى support@bugit.dev، علمًا بأن الدعم يُقدَّم باللغة الإنجليزية. هل تواجه مشكلة في الإعداد خلال أيامك السبعة الأولى؟ سنساعدك على تشغيله أو نساعدك في استرداد المبلغ من المتجر.

يرجى عدم تضمين معلومات المشروع السرية في رسائل الدعم

يختلف كل مشروع عن الآخر، وقد يكون قدر كبير من عملك سريًا. إذا أرسلت إلينا بريدًا إلكترونيًا، فيرجى وصف مشكلة BugIt بعبارات عامة فقط؛ ولا تلصق شيفرات سرية، أو تفاصيل تقارير أخطاء، أو مواصفات، أو لقطات شاشة، أو بيانات من مشروعك. لا تتحمل Taskivator مسؤولية أي معلومات سرية تُرسل إلينا، ويُحذف فورًا أي محتوى سري نتلقاه. كلما أمكن، اطلب المساعدة من مساعد الذكاء الاصطناعي أولًا؛ فهو يستطيع حل معظم المشكلات مباشرة داخل محررك.

شكرًا لاختيارك BugIt.

أنشئ تذاكر خلل واضحة، وتجنّب التكرارات، واستعد وقتك؛ خللًا تلو الآخر.

BugIt QA Agent · دليل الإعداد والاستخدام · © 2026 Taskivator. جميع الحقوق محفوظة. · bugit.dev · ملفا `LICENSE` و `PRIVACY.md` المرفقان مع التنزيل هما الوثيقتان الحاكمتان.